

OpenCV 颜色+形状识别 使用教程

适用平台: Windows PC + USB摄像头 (可移植 RK3588)

一、环境准备

1.1 安装 Python

下载 Python 3.10 或 3.11 (不要 3.12+) :

1.2 创建虚拟环境

打开 CMD, 进入你的项目环境:

```
conda activate <环境名>
```

1.3 安装依赖

```
pip install opencv-python==4.10.0.84 numpy      #cv功能  
pip install pyserial                          #串口功能
```

二、两个文件的关系

hsv_tuner.py | **校准工具** — 拖动滑条找颜色阈值 | USB 摄像头实时画面 + 6个滑条 |

| cv_color_shape.py | **检测脚本** — 识别颜色+形状+串口发送 | USB 摄像头实时画面 + 画框标注

工作流程:

```
hsv_tuner.py → 调出每种颜色的 HSV 阈值 → 填入 cv_color_shape.py 的  
COLOR_CONFIG → 跑检测
```

三、hsv_tuner.py — 阈值校准工具

3.1 启动

```
python hsv_tuner.py
```

会弹出两个窗口:

- "HSV Tuner" — 摄像头画面 + 6个滑条
- "Mask" — 黑白二值图（目标白色 = 识别到了，背景黑色 = 过滤掉了）

3.2 操作步骤（拿红色物体举例）

1. 把红色物体放在摄像头前，**保持和比赛时一样的光线**
2. 拖动 6 个滑条，顺序按 **V → S → H**:

第一步: 调 V（亮度）

- 拖动 **V_low** 从 0 往大拉，拉到背景刚好变黑
- 拖动 **V_high** 从 255 往小拉，拉到目标物体刚好变白
- 目标: Mask 窗口中目标纯白、背景纯黑

第二步: 调 S（饱和度）

- 拖动 **S_low** 往大拉，过滤掉白色墙壁、灰色桌面的噪点
- **S_high** 一般保持 255 不动

第三步: 调 H（色相）

- 收紧 **H_low** 和 **H_high**，只保留目标颜色的色相范围
- **红色特殊**: 红色在 HSV 色环两端（0~10 和 160~180），代码已自动处理，调好 0~10 段就行

3. 目标变纯白、背景变纯黑后，按键盘 **s** 保存
4. 终端会打印一行: `SAVED: (H_low, H_high, S_low, S_high, V_low, V_high)`
5. 同时写入 `thresholds.txt` 文件

3.3 逐个颜色校准

对每种颜色重复上面的操作:

- 红色、绿色、蓝色、黄色、紫色、黑色

黑色注意: V 通道留 0~60 就行, H 和 S 设为 0~180 和 0~255 不动。

3.4 退出

按 q 或直接关窗口。

四、cv_color_shape.py — 检测主脚本

4.1 填入校准好的阈值

打开 cv_color_shape.py, 找到 COLOR_CONFIG 字典。把 hsv_tuner.py 调出来的值填进去:

```
COLOR_CONFIG = {  
  
    "RED": {  
  
        "hsv_low": (0, 120, 80),          # 改成你调出的 (H_low, S_low, V_low)  
  
        "hsv_high": (10, 255, 255),      # 改成你调出的 (H_high, S_high, V_high)  
  
        "extra": ((160, 120, 80), (180, 255, 255)),  
  
        "draw": (0, 0, 255),  
  
    },  
  
    # ... 其他颜色同理  
  
}
```

4.2 配置检测目标

找到 DETECT 字典, 指定你要检测的颜色乘形状组合:

```
DETECT = {  
  
    "RED": ["RECT", "TRIANGLE"],          # 红色矩形 + 红色三角形
```

```

"GREEN":  ["CIRCLE", "TRIANGLE"],          # 绿色圆形 + 绿色三角形

"BLUE":   ["RECT", "CIRCLE", "TRIANGLE"],    # 蓝色全部三种形状

"YELLOW": [],                                # 跳过黄色

"PURPLE": [],                                # 跳过紫色

"BLACK":  [],                                # 跳过黑色

}

```

可选形状: "RECT" (矩形)、"CIRCLE" (圆形)、"TRIANGLE" (三角形)。空列表 = 跳过该颜色。

4.3 启动检测

```
python cv_color_shape.py
```

终端会先打印当前配置。

4.4 画面说明

OpenCV 窗口里:

- 矩形物体 → 画对应颜色的矩形框
- 圆形物体 → 画对应颜色的圆环
- 三角形物体 → 画对应颜色的轮廓线
- 左上角显示颜色名+形状名 (如 "RED RECT")

4.5 按键操作

| q | 退出程序 | 终端窗口 |

| ESC | 退出程序 | OpenCV画面窗口 |

| s | 截图保存 snapshot.jpg | 终端窗口 |

4.6 串口发送 (发给 STM32)

文件顶部有串口开关:

```
COM_PORT = 'COM3'      # 改成你实际的串口号（设备管理器里看）

BAUDRATE = 115200      # 和 STM32 一致

USE_UART = False       # True = 启用串口, False = 只用画面
```

改为 USE_UART = True 后，检测到的每个目标的几何中心坐标会通过串口发送，格式：

```
0,320,240;1,150,380\n
```

数字 0、1 是 DETECT 组合编号（终端启动时会打印对应关系）。分号分隔多个目标，换行结尾。

STM32 端解析（C 语言）：

```
char *token = strtok(buf, ";");

while (token) {

    int id, cx, cy;

    sscanf(token, "%d,%d,%d", &id, &cx, &cy);

    // 根据 id 查表得到颜色和形状

    token = strtok(NULL, ";");

}
```

五、完整工作流程

第1步：python hsv_tuner.py

对每种颜色拖动滑条

按 **s** 保存阈值

记下终端打印的六个数字

第2步：打开 `cv_color_shape.py`

把调好的六个数字填入 `COLOR_CONFIG`

在 `DETECT` 里勾选要检测的颜色+形状

`USE_UART = False`（先不接硬件）

第3步：`python cv_color_shape.py`

验证识别效果

如果某颜色不准，回第1步重新调

第4步：确认无误后

接上 `STM32`

`pip install pyserial`

`USE_UART = True`

重新运行

六、常见问题

Q1: 按 q 没反应？

在终端黑窗（PowerShell/CMD）里按 q，不要在 OpenCV 画面上按。如果还是不行，鼠标点一下 OpenCV 画面窗口让它获得焦点，再按 ESC。

Q2: 颜色识别不准

1. 回到 `hsv_tuner.py` 重新调该颜色的阈值
2. 光线变了阈值就不准，找比赛场地相同光线下校准
3. 摄像头自动曝光/白平衡导致的漂移。如果摄像头 UVC 驱动支持：

```
cap = cv2.VideoCapture(0)

cap.set(cv2.CAP_PROP_AUTO_WB, 0)      # 关自动白平衡

cap.set(cv2.CAP_PROP_AUTO_EXPOSURE, 0) # 关自动曝光

cap.set(cv2.CAP_PROP_EXPOSURE, -6)    # 手动固定曝光
```

如果 `set()` 返回 `False`，说明驱动不支持，只能把阈值拉宽来补偿。

Q3: 三角形识别不稳定

三角形检测用的是多边形逼近 (`approxPolyDP`)。手工画的歪三角可能轮廓顶点数不对。在 `classify_shape` 函数里降低逼近精度参数 `0.03` 改成 `0.02`，或者把圆度判断范围 `0.65~0.78` 放宽到 `0.55~0.85`。

Q4: 画面卡 / FPS 低

1. 把 `DETECT` 里不需要的颜色设为空列表（跳过 `mask` 计算，省 CPU）
2. 降低摄像头分辨率：

```
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)

cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

Q5: USB 摄像头适用性

完全适用。HSV 是后处理数学变换，和摄像头硬件无关。但要关闭自动曝光和白平衡（见 Q2）。廉价 USB 摄像头如果 UVC 驱动不支持这些参数，唯一办法就是调宽阈值范围。

Q6: Python 版本

必须 3.10 或 3.11。3.12+ 上 OpenCV 4.10 有零星 DLL 加载问题。在 Python 3.9 上也能跑但不是首选。查看当前版本:

```
python --version
```